Fuzzycomplete

Fuzzy SQL completion with a catalog-aware matcher

Aa

Install & load

```
SET allow_extension_repositories = 'allowed';
CREATE EXTENSION REPOSITORY IF NOT EXISTS query_farm WITH PREFIX 'https://duckdb-extensions.query.farm';
INSTALL fuzzycomplete FROM query_farm;  LOAD fuzzycomplete FROM query_farm;
```

01 At a glance

Suggest SQL keywords and catalog names from a partial query using a fuzzy matcher. Loading it replaces the CLI's preloaded stock autocomplete; inspect suggestions directly through the table function.

Partial SQL → Catalog + matcher → Suggestions

02 Completion surface

`sql_auto_complete(query)`

Table function: suggest completions for partial SQL text.

`suggestion`

Candidate text returned by the completion provider.

`suggestion_start`

Character offset at which the candidate should replace text.

`max_suggestion_count := 5`

Cap the number of suggestions; stock named parameters are accepted.

`Tab in the CLI`

Requests completion through the active provider.

`duckdb_extensions()`

Check whether autocomplete was already loaded.

`duckdb_functions()`

Inspect the registered completion function and its parameters.

03 Keep in mind

Check the active provider

LOAD fuzzycomplete replaces an already loaded stock provider, so load it after startup or in ~/.duckdbrc. If you load both explicitly, load autocomplete first. Check with SLCT: the fuzzy matcher suggests SELECT. loaded = true alone does not identify the provider.

04 Inspect the loaded provider

```
SELECT extension_name, loaded
FROM duckdb_extensions()
WHERE extension_name IN
  ('autocomplete', 'fuzzycomplete');
```

Reports loaded extensions; use the suggestion check below to verify fuzzy behavior.

05 Inspect SQL suggestions

```
SELECT trim(suggestion) AS candidate
FROM sql_auto_complete('SLCT');
-- SELECT
```

A subsequence query confirms that the fuzzy provider is active.

06 Try a catalog abbreviation

MANUAL SETUP

```
CREATE TEMP TABLE getNotificationStatus(
  id INTEGER
);
-- In a compatible interactive CLI, type:
-- SELECT * FROM gnst
-- Then press Tab before submitting.
```

The client must call DuckDB's active completion provider for Tab.

07 In practice

Matcher and client are separate

Fuzzy matching rewards ordered characters and word boundaries; ties favour SELECT and FROM, your own tables, and columns of tables in the statement. Comments are skipped. SQL calls inspect suggestions; they do not add an editor to other clients.

Full docs & examples ↗ Source ↗